

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
DEEP LEARNING

ĐỀ TÀI:

ỨNG DỤNG MẠNG NƠ-RON TÍCH CHẬP (CNN)
VÀ DEEPSORT TRONG HỆ THỐNG NHẬN DIỆN,
THEO DÕI VÀ CẢNH BÁO ĐEO KHẨU TRANG
THỜI GIAN THỰC

Giảng viên hướng dẫn: ThS. Huỳnh Thái Học

TP. HỒ CHÍ MINH, NĂM 2026

TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
DEEP LEARNING

ĐỀ TÀI:

ỨNG DỤNG MẠNG NƠ-RON TÍCH CHẬP (CNN)
VÀ DEEPSORT TRONG HỆ THỐNG NHẬN DIỆN,
THEO DÕI VÀ CẢNH BÁO ĐEO KHẨU TRANG
THỜI GIAN THỰC

Giảng viên hướng dẫn: ThS. Huỳnh Thái Học
Sinh viên thực hiện: Trần Thị Kim Ngân – 2001230554
Phạm Nguyễn Hồng Ngọc
Trần Quốc Trường – 2001231017
Nguyễn Duy Hưng

TP. HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm 11 xin gửi lời cảm ơn chân thành đến Ban giám hiệu Trường Đại học Công thương TP.HCM cùng Khoa Công nghệ Thông tin đã tạo môi trường học tập tốt nhất cho chúng em trong suốt thời gian qua.

Đặc biệt, nhóm xin gửi lời tri ân sâu sắc đến ThS. Huỳnh Thái Học – người thầy đã trực tiếp hướng dẫn, dành nhiều thời gian tâm huyết để dẫn dắt nhóm qua các buổi thực hành. Nhờ có sự định hướng tỉ mỉ của các thầy, nhóm đã tiếp cận môn học dưới góc nhìn toàn diện, nâng cao tư duy thực chiến và hiểu rõ hơn tính thiết thực của kiến thức vào thực tế.

Do giới hạn về mặt thời gian và trải nghiệm thực tế, bài báo cáo khó lòng hoàn hảo tuyệt đối. Nhóm 11 rất hy vọng sẽ nhận được sự lượng thứ cùng những nhận xét, đóng góp quý báu từ thầy để hoàn thiện năng lực của bản thân trong tương lai.

Kính chúc các thầy luôn mạnh khỏe, hạnh phúc và thành công trên con đường giáo dục!

Nhóm 11 xin chân thành cảm ơn!

(Tập thể thành viên nhóm)

Trần Thị Kim Ngân – 2001230554

Phạm Nguyễn Hồng Ngọc – 2001230554

Trần Quốc Trường – 2001231017

Nguyễn Duy Hưng – 2001230554

MỤC LỤC

DANH MỤC BẢNG BIỂU	4
1 CHƯƠNG 1: MỞ ĐẦU	7
1.1 Lý do chọn đề tài	7
1.2 Mục tiêu nghiên cứu	7
1.2.1 Mục tiêu tổng quát	7
1.2.2 Mục tiêu cụ thể	7
2 CHƯƠNG 2: TỔNG QUAN CƠ SỞ LÝ THUYẾT	9
2.1 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)	9
2.1.1 Feature trong CNN	9
2.1.2 Những lớp cơ bản của mạng CNN	9
2.2 Thuật toán theo dõi đối tượng đa mục tiêu DeepSORT	10
2.2.1 Bài toán theo dõi đối tượng (Object Tracking)	10
2.2.2 Bộ lọc Kalman (Kalman Filter)	10
2.2.3 Thuật toán Hungary (Hungarian Algorithm)	11
2.2.4 Mạng nhúng sâu (Deep Appearance Descriptor)	11
3 CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG	12
3.1 Kiến trúc tổng quan	12
3.2 Xây dựng và tiền xử lý bộ dữ liệu (dataset)	13
3.2.1 Thu thập dữ liệu	13
3.2.2 Gán nhãn dữ liệu	14
3.3 Quá trình huấn luyện mô hình CNN	15
3.3.1 Môi trường cấu hình	15
3.3.2 Kiến trúc mô hình MaskCNN	16
3.3.3 Tham số huấn luyện	17
3.4 Triển khai module theo dõi và cấu hình logic cảnh báo	18
3.4.1 Tích hợp CNN với Deepsort	18
3.4.2 Xây dựng thuật toán đếm thời gian (Timer) theo ID đối tượng . .	19
3.4.3 Cơ chế kích hoạt cảnh báo	20
4 TÀI LIỆU THAM KHẢO	22

DANH MỤC BẢNG BIỂU

3.1	Các giai đoạn xử lý tuần tự trong kiến trúc pipeline của hệ thống	12
3.2	Cấu trúc phân lớp và mô tả các nhãn trong bộ dữ liệu huấn luyện	14
3.3	Các kỹ thuật tăng cường dữ liệu và cấu hình tham số áp dụng cho tập huấn luyện	15
3.4	Cấu hình phần cứng và phần mềm môi trường huấn luyện	16
3.5	Cấu trúc chi tiết các lớp và khối chức năng trong mô hình MaskCNN . .	17
3.6	Cấu hình tham số siêu tham số (Hyperparameters) trong quá trình huấn luyện	18
3.7	Nguyên tắc hoạt động và xử lý logic của bộ đếm thời gian (Timer)	20
3.8	Các thành phần hiển thị trực quan và cấu hình màu sắc trên giao diện cảnh báo	21

LỜI MỞ ĐẦU

Trong những năm gần đây, Trí tuệ nhân tạo (AI) đã và đang khẳng định vai trò cốt lõi trong cuộc cách mạng công nghệ toàn cầu. Sự bùng nổ về năng lực tính toán của phần cứng (GPU) cùng với lượng dữ liệu số khổng lồ được thu thập đã đưa Học máy (Machine Learning) tiến một bước dài, tạo tiền đề cho sự ra đời của Học sâu (Deep Learning). Các mô hình Học sâu đã giúp máy tính thực thi được những tác vụ vô cùng phức tạp với độ chính xác vượt trội, từ phân loại hàng ngàn vật thể, xử lý ngôn ngữ tự nhiên cho đến các hệ thống tự hành. Nhờ đó, việc ứng dụng Học sâu vào các hệ thống camera giám sát thông minh (Smart Surveillance) đã trở thành một xu hướng tất yếu trong kỷ nguyên số.

Trên thực tế, công tác đảm bảo an ninh, trật tự tại các khu vực đặc thù và có tính chất rủi ro cao như hệ thống ngân hàng, phòng giao dịch hay các điểm rút tiền tự động (ATM) luôn là bài toán thách thức đối với các nhà quản lý. Đây là những nơi thường xuyên diễn ra các giao dịch tài chính lớn, dễ trở thành mục tiêu hàng đầu của tội phạm cướp giật, trộm cắp tài sản. Thủ đoạn phổ biến của các đối tượng này là cố tình che giấu nhân dạng bằng cách đeo khẩu trang, đội mũ kín hoặc che mặt trước khi bước vào khu vực giao dịch nhằm vô hiệu hóa khả năng nhận diện của hệ thống camera truyền thống. Mặc dù các camera hiện nay có mật độ dày đặc, nhưng chúng chỉ hoạt động theo cơ chế ghi hình thụ động và phụ thuộc hoàn toàn vào sức người để theo dõi màn hình, dẫn đến việc khó phát hiện và ngăn chặn kịp thời các hành vi nghi vấn diễn ra trong thời gian ngắn.

Để giải quyết triệt để hạn chế trên, việc xây dựng một hệ thống giám sát chủ động, có khả năng tự động phân tích hành vi che mặt và đưa ra cảnh báo tức thời là vô cùng cấp thiết. Xuất phát từ nhu cầu thực tiễn đó, nhóm 11 đã tập trung nghiên cứu, phát triển đề tài "Ứng dụng mạng nơ-ron tích chập (CNN) và DeepSORT trong hệ thống nhận diện, theo dõi và cảnh báo đeo khẩu trang thời gian thực". Điểm đột phá của đề tài là việc kết hợp mô hình thị giác máy tính dựa trên mạng nơ-ron tích chập (CNN) để phát hiện trạng thái khuôn mặt, kết hợp với thuật toán theo dõi đa mục tiêu DeepSORT nhằm duy trì định danh, định vị quỹ đạo và thiết lập logic đếm thời gian. Hệ thống sẽ tự động kích hoạt tín hiệu cảnh báo đến bộ phận an ninh ngay khi phát hiện có đối tượng cố tình che mặt liên tục vượt quá khung thời gian an toàn quy định.

Nội dung chính của báo cáo đồ án được kết cấu thành 4 chương như sau:

- **Chương 1: Tổng quan về đề tài** – Giới thiệu bối cảnh, lý do chọn đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi tiếp cận của hệ thống.

- **Chương 2: Cơ sở lý thuyết** – Trình bày nền tảng khoa học về mạng nơ-ron tích chập (CNN) trong bài toán phát hiện vật thể, nguyên lý hoạt động của thuật toán theo dõi đối tượng DeepSORT và cơ chế lọc Kalman.
- **Chương 3: Thiết kế và thực nghiệm hệ thống** – Mô tả quy trình thu thập, tiền xử lý dữ liệu, quá trình huấn luyện mô hình và xây dựng luồng xử lý logic cảnh báo theo thời gian thực qua Webcam.
- **Chương 4: Kết quả và phương hướng phát triển** – Đánh giá hiệu năng thực nghiệm của mô hình qua các chỉ số kỹ thuật, nhìn nhận những mặt đạt được, hạn chế và đề xuất giải pháp tối ưu hệ thống trong tương lai.

1 CHƯƠNG 1: MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong bối cảnh an ninh tại các ngân hàng và trung tâm tài chính ngày càng trở nên phức tạp, việc phát hiện và ngăn chặn kịp thời các đối tượng cố tình che giấu nhân dạng (đeo khẩu trang, kính râm, mũ kín) là rất cần thiết. Tuy nhiên, các hệ thống camera giám sát hiện nay chủ yếu chỉ ghi hình thụ động, thiếu khả năng phân tích và cảnh báo chủ động thời gian thực. Nhân viên an ninh khó có thể theo dõi liên tục nhiều màn hình cùng lúc, dẫn đến bỏ sót các hành vi đáng ngờ.

Với sự phát triển mạnh mẽ của Trí tuệ nhân tạo, đặc biệt là Thị giác máy tính, việc áp dụng Mạng nơ-ron tích chập (CNN) để nhận diện trạng thái đeo khẩu trang và DeepSORT để theo dõi, định danh và tính toán thời gian che mặt của đối tượng đã trở thành giải pháp hiệu quả. Khi thời gian che mặt vượt quá ngưỡng cho phép (ví dụ: 20 giây), hệ thống sẽ tự động kích hoạt cảnh báo giúp lực lượng an ninh kịp thời can thiệp.

Xuất phát từ nhu cầu thực tiễn nâng cao năng lực giám sát an ninh chủ động tại các tổ chức tín dụng, nhóm quyết định thực hiện đề tài: “Ứng dụng Mạng nơ-ron tích chập (CNN) và DeepSORT trong hệ thống nhận diện, theo dõi và cảnh báo đeo khẩu trang thời gian thực”.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và xây dựng thành công một hệ thống giám sát an ninh thông minh có khả năng tự động nhận diện hành vi đeo khẩu trang/che mặt, theo dõi quỹ đạo di chuyển của đối tượng trong thời gian thực và tự động phát tín hiệu cảnh báo đến bộ phận quản lý khi đối tượng vi phạm quy định về thời gian an toàn tại các khu vực trọng yếu như ngân hàng.

1.2.2 Mục tiêu cụ thể

- **Về mặt lý thuyết:** Nghiên cứu và nắm vững nguyên lý hoạt động của các kiến trúc mạng nơ-ron tích chập (CNN) trong bài toán phát hiện vật thể (Object Detection); làm chủ thuật toán theo dõi đối tượng DeepSORT ứng dụng bộ lọc Kalman và mạng nhúng sâu (Deep Appearance Descriptor).
- **Về mặt dữ liệu:** Thu thập, chuẩn hóa và gán nhãn bộ dữ liệu hình ảnh (Dataset) chất lượng cao bao gồm các trạng thái: đeo khẩu trang, không đeo khẩu trang, và các hành vi che mặt khác phù hợp với bối cảnh camera giám sát góc cao tại ngân hàng.

- **Về mặt công nghệ:** Huấn luyện thành công mô hình CNN đạt độ chính xác cao ($Accuracy \geq 90\%$), tốc độ xử lý nhanh (FPS đáp ứng thời gian thực) trên các luồng video livestream trực tiếp. Tích hợp module DeepSORT để duy trì ID nhân dạng ổn định, không bị mất dấu ngay cả khi đối tượng bị che khuất tạm thời.
- **Về mặt tính năng cảnh báo:** Phát triển thuật toán logic đếm thời gian (Timer) dựa trên ID của DeepSORT. Thiết lập hệ thống kích hoạt âm thanh/cảnh báo hiển thị trực quan khi một ID duy trì trạng thái che mặt liên tục quá thời gian cấu hình (ví dụ: 20 giây).
- **Về mặt ứng dụng và thực nghiệm:** Đánh giá hiệu năng của mô hình thông qua môi trường mô phỏng bằng webcam thời gian thực; bước đầu phân tích, đo lường các chỉ số sai số (như tỷ lệ nhận diện sai, tỷ lệ báo động giả - False Alarm Rate) trong không gian phòng thí nghiệm, từ đó đề xuất định hướng tối ưu phần cứng và giải pháp triển khai hệ thống thực tế trong tương lai.

2 CHƯƠNG 2: TỔNG QUAN CƠ SỞ LÝ THUYẾT

2.1 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)

Mạng nơ-ron tích chập (CNN) là một mô hình học sâu (*deep learning*) được sử dụng rộng rãi để xử lý dữ liệu dạng hình ảnh, dựa trên cơ chế tổ chức sắp xếp của vỏ não thị giác ở động vật [1]. Mục tiêu thiết kế cốt lõi của kiến trúc này là tự động học hỏi một cách linh hoạt các cấu trúc phân cấp không gian của các đặc trưng, từ các dạng mẫu cấp thấp đến cấp cao. Mô hình này đã chứng minh được hiệu năng vượt trội trong nhiều tác vụ khác nhau, bao gồm nhận diện khuôn mặt, định danh vật thể và phát hiện biến báo giao thông – đặc biệt là trong lĩnh vực robot và xe tự hành. Việc giảm thiểu số lượng tham số so với mạng nơ-ron nhân tạo (ANN) truyền thống là một trong những đặc điểm quan trọng nhất của CNN [1].

Mục đích chính của CNN là trích xuất và học các đặc trưng có giá trị từ dữ liệu đầu vào. Trong quy trình này, các lớp khởi đầu hoạt động như một tập hợp các bộ trích xuất đặc trưng tích chập (*convolutional feature extractors*) dựa trên các bộ lọc (*filters*) có khả năng tự học hỏi thông số. Các bộ lọc này hoạt động như một cửa sổ trượt (*sliding window*) di chuyển liên tục trên từng vùng của dữ liệu hình ảnh đầu vào. Trong đó, khoảng cách dịch chuyển chồng lấp giữa các bước trượt được gọi là bước nhảy (*stride*), và kết quả đầu ra thu được gọi là các bản đồ đặc trưng (*feature maps*).

Các lõi tích chập (*convolutional kernels*) cấu thành nên mỗi lớp trong mạng CNN và được sử dụng để tạo ra các bản đồ đặc trưng khác nhau. Các vùng nơ-ron lân cận sẽ được kết nối với một nơ-ron cụ thể thuộc bản đồ đặc trưng của lớp kế tiếp. Để tạo ra một bản đồ đặc trưng, lõi tích chập phải được chia sẻ trọng số trên toàn bộ các vị trí không gian của đầu vào. Sau khi các lớp tích chập (*convolutional layers*) và lớp nén/gộp (*pooling layers*) được thiết lập, một hoặc nhiều lớp kết nối đầy đủ (*fully connected layers*) sẽ được sử dụng ở giai đoạn cuối để hoàn thiện tác vụ phân loại [1].

2.1.1 Feature trong CNN

Các feature (đặc trưng) được sử dụng để trích xuất thông tin từ ảnh đầu vào. Các feature này được học tự động từ dữ liệu và được sử dụng để giảm thiểu số lượng thông tin cần xử lý trong quá trình huấn luyện mạng. Các feature này có thể là các đường cạnh, các vùng sáng tối, các đường cong và các đối tượng phức tạp hơn. Các feature này được trích xuất thông qua các bộ lọc tích chập và được kết hợp lại để tạo thành các feature maps, đóng vai trò quan trọng trong việc phân loại ảnh.

2.1.2 Những lớp cơ bản của mạng CNN

Lớp tích chập (Convolutional Layer): Đây là thành phần quan trọng nhất của toàn bộ mạng CNN. Các yếu tố cốt lõi trong lớp này bao gồm: filter (bộ lọc), stride (bước dịch),

padding (phần đệm) và feature map (bản đồ đặc trưng).

- **Filter (Kernel):** Mạng CNN sử dụng các filter để áp dụng vào từng vùng của ma trận hình ảnh. Các filter thường là một ma trận trọng số hai chiều (2D) giúp phát hiện các đặc trưng trong hình ảnh.[2]
- **Stride:** Là bước di chuyển của kernel trên ma trận đầu vào, xác định khoảng cách giữa các lần quét. Stride lớn sẽ tạo ra đầu ra nhỏ hơn.[2]
- **Padding:** Được sử dụng khi kích thước của filter không khớp với kích thước của hình ảnh đầu vào.[2]
- **Feature map:** Ma trận kết quả thu được sau khi filter quét qua toàn bộ ma trận ảnh đầu vào. Đóng vai trò lưu giữ, trích xuất và làm nổi bật các đặc điểm quan trọng (như đường nét, màu sắc, hình khối) từ dữ liệu đầu vào.

Activation Layer: Sau khi thực hiện phép tích chập, dữ liệu sẽ đi qua lớp kích hoạt để thêm tính phi tuyến vào mô hình. Hàm kích hoạt phổ biến nhất là ReLU (Rectified Linear Unit).

Pooling Layer: Hay còn gọi là downsampling, có nhiệm vụ giảm chiều dữ liệu và giảm số lượng tham số trong đầu vào.[2]

Fully-connected Layer: Mỗi đầu vào được liên kết với đầu ra thông qua các trọng số có thể học được. Thông thường, lớp fully connected cuối cùng sẽ được theo sau bởi một bộ phân loại Softmax.[1]

2.2 Thuật toán theo dõi đối tượng đa mục tiêu DeepSORT

2.2.1 Bài toán theo dõi đối tượng (Object Tracking)

Bài toán theo dõi đối tượng đa mục tiêu (Multiple Object Tracking - MOT) là nhiệm vụ phát hiện nhiều đối tượng trong video hoặc hình ảnh, dự đoán quỹ đạo chuyển động và duy trì danh tính (ID) nhất quán qua các khung hình, ngay cả khi chúng có thể bị che khuất hoặc tạm thời biến mất khỏi tầm nhìn. Để đạt được điều này, các thuật toán theo dõi đa đối tượng thường sử dụng sự kết hợp của các kỹ thuật phát hiện đối tượng, liên kết dữ liệu và dự đoán quỹ đạo. Các thuật toán này cũng phải giải quyết các thách thức như xử lý sự che khuất, duy trì nhận dạng đối tượng trong các cảnh đông đúc và thích ứng với những thay đổi về hình dạng hoặc chuyển động của đối tượng.

2.2.2 Bộ lọc Kalman (Kalman Filter)

Bộ lọc Kalman là thành phần cốt lõi để mô hình hóa chuyển động của đối tượng trong không gian hai chiều. Nó dự đoán vị trí và vận tốc của đối tượng ở khung hình tiếp theo

dựa trên trạng thái trước đó. Trong DeepSORT, không gian trạng thái thường là một vector cấu trúc 8 chiều:

$$x = (u, v, a, h, u', v', a', h')$$

Trong đó (u, v) là tọa độ trung tâm, a là tỷ lệ khung hình, h là chiều cao, và các ký hiệu có dấu phẩy (u', v', a', h') biểu thị các vận tốc tương ứng. Bộ lọc Kalman thực hiện hai bước chính là **Predict** (Dự đoán vị trí mới) và **Update** (Cập nhật dữ liệu khi thu được đo lường thực tế từ module detection). Nó giúp duy trì quỹ đạo ổn định và xử lý hiệu quả các trường hợp đối tượng tạm thời bị che khuất ngắn hạn.

2.2.3 Thuật toán Hungary (Hungarian Algorithm)

Thuật toán Hungarian dùng để giải bài toán gán tối ưu. Nó ghép nối các detection mới với các track đang tồn tại sao cho tổng chi phí (được tính bằng khoảng cách Mahalanobis và khoảng cách Cosine) là nhỏ nhất.

2.2.4 Mạng nhúng sâu (Deep Appearance Descriptor)

Deep Appearance Descriptor là một mạng CNN được huấn luyện để trích xuất vector embedding mô tả ngoại hình của đối tượng nhằm hỗ trợ nhận diện lại đối tượng khi bị che khuất ngắn hạn.

3 CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG

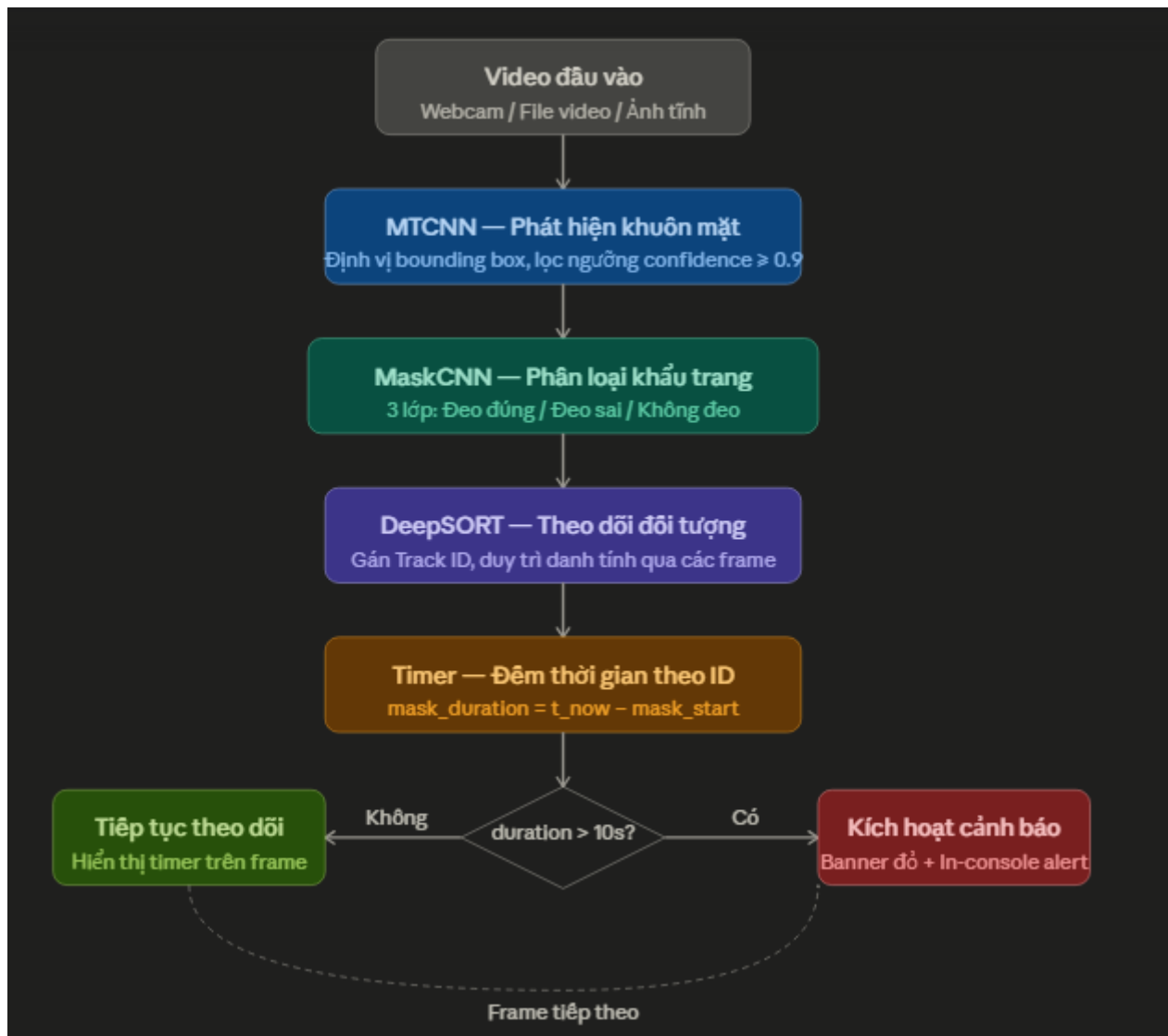
3.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo kiến trúc pipeline đa tầng, kết hợp các kỹ thuật học sâu (*deep learning*) với thuật toán theo dõi đối tượng thời gian thực. Luồng xử lý chính bao gồm năm giai đoạn tuần tự được mô tả chi tiết trong Bảng 3.1.

Bảng 3.1: Các giai đoạn xử lý tuần tự trong kiến trúc pipeline của hệ thống

Giai đoạn	Module	Chức năng
1	Video đầu vào	Nhận luồng video từ webcam, file video (.mp4) hoặc ảnh tĩnh thông qua OpenCV (<code>cv2.VideoCapture</code>).
2	MTCNN	Phát hiện và định vị khuôn mặt trong từng frame, lọc theo ngưỡng confidence ≥ 0.9 , trả về bounding box (x_1, y_1, x_2, y_2) .
3	MaskCNN	Phân loại khuôn mặt vừa phát hiện thành một trong ba trạng thái: Đeo đúng (<code>with_mask</code>), Đeo sai (<code>mask_wearred_incorrect</code>), Không đeo (<code>without_mask</code>).
4	DeepSORT	Gán và duy trì Track ID nhất quán cho từng người qua nhiều frame liên tục, sử dụng phương pháp kết hợp IoU và <i>appearance embedding</i> .
5	Timer & Alert	Đếm tích lũy thời gian đeo khẩu trang đúng cách theo từng Track ID; kích hoạt cảnh báo trực quan khi vượt ngưỡng 10 giây.

Khi phát hiện đối tượng vi phạm (đeo khẩu trang đúng cách quá 10 giây mà hệ thống muốn cảnh báo, hoặc không đeo), hệ thống hiển thị banner cảnh báo màu đỏ trực tiếp lên frame video với thông tin ID người dùng và thời gian vi phạm. Sơ đồ luồng xử lý chi tiết được thể hiện trong Hình 3.1.



3.2 Xây dựng và tiền xử lý bộ dữ liệu (dataset)

3.2.1 Thu thập dữ liệu

Bộ dữ liệu được sử dụng trong đề tài là *Face Mask Dataset* (FMD_DATASET) do Shiekh Burhan công bố trên nền tảng Kaggle (kaggle.com/datasets/shiekhburhan/face-mask-dataset). Dataset được tải về tự động trong môi trường Google Colab bằng Kaggle API thông qua dòng lệnh sau:

```
!kaggle datasets download -d shiekhburhan/face-mask-dataset -p /content/datas
```

Dataset được tổ chức theo cấu trúc thư mục chuẩn ImageFolder của PyTorch với ba nhãn lớp tương ứng ba trạng thái đeo khẩu trang, chi tiết cấu trúc được thể hiện trong Bảng 3.2.

Bảng 3.2: Cấu trúc phân lớp và mô tả các nhãn trong bộ dữ liệu huấn luyện

Tên thư mục (Class)	Nhãn hiển thị	Mô tả
mask_wearred_incorrect	Đeo sai (Class 0)	Người đeo khẩu trang không đúng cách: hở mũi, hở cằm, đeo lệch hoặc che mặt bằng vật khác.
with_mask	Đeo đúng (Class 1)	Người đeo khẩu trang đúng cách, che kín mũi và miệng.
without_mask	Không đeo (Class 2)	Người không đeo khẩu trang, khuôn mặt hoàn toàn lộ ra.

Để đảm bảo cân bằng dữ liệu trong quá trình huấn luyện, hệ thống áp dụng hàm `limit_dataset_per_class()` để giới hạn số lượng ảnh tối đa mỗi lớp không vượt quá 3.500 mẫu. Nếu một lớp có nhiều hơn ngưỡng này, dữ liệu sẽ được lấy mẫu ngẫu nhiên (*random sampling*) để tránh hiện tượng mất cân bằng lớp. Toàn bộ dataset được chia theo tỷ lệ 70-15-15 bao gồm:

- **70% — Tập huấn luyện (Training set):** dùng để cập nhật tham số mô hình.
- **15% — Tập kiểm định (Validation set):** đánh giá mô hình sau mỗi epoch.
- **15% — Tập kiểm tra (Test set):** đánh giá cuối cùng sau khi hoàn thành huấn luyện.

3.2.2 Gán nhãn dữ liệu

Dataset FMD_DATASET đã được gán nhãn sẵn thông qua cấu trúc thư mục theo chuẩn ImageFolder. Mỗi thư mục con tương ứng với một nhãn lớp; PyTorch tự động ánh xạ tên thư mục sang chỉ số số nguyên theo thứ tự alphabet thông qua thuộc tính `full_dataset.class_to_idx` như sau:

```
full_dataset.class_to_idx:
'mask_wearred_incorrect' -> 0
'with_mask'              -> 1
'without_mask'           -> 2
```

Các kỹ thuật tăng cường dữ liệu (*Data Augmentation*) được áp dụng riêng cho tập huấn luyện nhằm tăng tính đa dạng và khả năng tổng quát hóa của mô hình, tránh hiện tượng quá khớp (*overfitting*). Chi tiết các cấu hình tham số được mô tả cụ thể trong Bảng 3.3.

Bảng 3.3: Các kỹ thuật tăng cường dữ liệu và cấu hình tham số áp dụng cho tập huấn luyện

Kỹ thuật Augmentation	Tham số	Mục đích
Resize	128×128 pixels	Chuẩn hóa kích thước đầu vào đồng nhất.
RandomHorizontalFlip	$p = 0.5$ (mặc định)	Tăng cường biến thể góc nhìn ngang của khuôn mặt.
RandomRotation	$\pm 15^\circ$	Mô phỏng các trạng thái khuôn mặt nghiêng.
ColorJitter	brightness = 0.2, contrast = 0.2, saturation = 0.2	Mô phỏng điều kiện ánh sáng khác nhau trong thực tế giám sát.
Normalize (ImageNet)	mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]	Chuẩn hóa phân phối pixel về cùng thang đo giúp mô hình hội tụ nhanh hơn.

Tập *validation* và *test* chỉ áp dụng hai kỹ thuật thiết yếu là *Resize* và *Normalize*, hoàn toàn không sử dụng các kỹ thuật *augmentation* biến đổi hình học hay màu sắc. Điều này đảm bảo tính khách quan và trung thực khi đánh giá hiệu năng thực tế của mô hình trên những tập dữ liệu chưa từng được nhìn thấy trước đó.

3.3 Quá trình huấn luyện mô hình CNN

3.3.1 Môi trường cấu hình

Quá trình huấn luyện được thực hiện trên nền tảng Google Colaboratory (Google Colab) với GPU Tesla T4 do Google cung cấp miễn phí. Tập dữ liệu được lưu trữ trên Google Drive và mount vào Colab runtime thông qua API:

```
from google.colab import drive
drive.mount('/content/drive')
```

Cấu hình phần cứng và phần mềm môi trường huấn luyện được thống kê chi tiết trong Bảng 3.4:

Bảng 3.4: Cấu hình phần cứng và phần mềm môi trường huấn luyện

Thành phần	Chi tiết
Nền tảng	Google Colaboratory (Colab).
GPU	NVIDIA Tesla T4 (15 GB VRAM).
CPU	Intel Xeon (2 vCPU).
RAM	12.7 GB.
Ngôn ngữ lập trình	Python 3.10+.
Deep Learning Framework	PyTorch 2.4+ / TorchVision 0.19+.
Face Detection	facenet-pytorch (MTCNN, InceptionResnetV1).
Object Tracking	deep-sort-realtime.
Xử lý ảnh / Video	OpenCV (cv2), Pillow (PIL).
Khoa học dữ liệu	NumPy 2.x, Matplotlib, Seaborn, scikit-learn.

Toàn bộ thư viện được cài đặt bằng pip tại thời điểm khởi động runtime:

```
!pip install torch torchvision opencv-python pillow
!pip install deep-sort-realtime facenet-pytorch soundfile
```

3.3.2 Kiến trúc mô hình MaskCNN

Mô hình phân loại khẩu trang MaskCNN được xây dựng từ đầu (*from scratch*) bằng PyTorch, gồm bốn khối tích chập (*Convolutional Block*) tuần tự, một lớp *Global Average Pooling* và ba lớp *Fully Connected*. Chi tiết cấu trúc từng tầng được thống kê cụ thể trong Bảng 3.5.

Bảng 3.5: Cấu trúc chi tiết các lớp và khối chức năng trong mô hình MaskCNN

Layer	Operation	Output Shape	Filters / Units
Input Layer	Raw RGB Image	$128 \times 128 \times 3$	3 channels
Conv Block 1	Conv2D ($3 \rightarrow 32$) + BN + ReLU + MaxPool(2) + Dropout(0.25)	$64 \times 64 \times 32$	32 filters
Conv Block 2	Conv2D ($32 \rightarrow 64$) + BN + ReLU + MaxPool(2) + Dropout(0.25)	$32 \times 32 \times 64$	64 filters
Conv Block 3	Conv2D ($64 \rightarrow 128$) + BN + ReLU + MaxPool(2) + Dropout(0.25)	$16 \times 16 \times 128$	128 filters
Conv Block 4	Conv2D ($128 \rightarrow 192$) + BN + ReLU + MaxPool(2) + Dropout(0.3)	$8 \times 8 \times 192$	192 filters
Global Average Pooling	AdaptiveAvgPool2d(1,1)	$1 \times 1 \times 192$	-
Fully Connected	Flatten + Linear($192 \rightarrow 128$) + BN + ReLU + Dropout(0.5) Linear($128 \rightarrow 64$) + BN + ReLU + Dropout(0.4) Linear($64 \rightarrow 3$)	3	3 classes
Output Layer	3-class probabilities	3	Softmax (tại inference)

Với input kích thước 128×128 pixels, kích thước bản đồ đặc trưng (*feature map*) thu nhỏ dần qua mỗi lớp *MaxPool*: $64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$ trước khi đi qua tầng *GAP* (*Global Average Pooling*) và các lớp kết nối đầy đủ *Fully Connected* (*FC*).

3.3.3 Tham số huấn luyện

Các tham số và cấu hình cụ thể áp dụng cho quá trình huấn luyện mô hình được thống kê chi tiết trong Bảng 3.6.

Bảng 3.6: Cấu hình tham số siêu tham số (Hyperparameters) trong quá trình huấn luyện

Tham số	Giá trị / Cấu hình
Số epochs tối đa	10 (có Early Stopping, patience=5)
Batch size	32
Optimizer (chính)	Adam (lr=0.001, weight_decay=5e-4)
Optimizer (so sánh)	SGD (lr=0.01, momentum=0.9, weight_decay=1e-4)
Loss function	CrossEntropyLoss với label_smoothing=0.1
Learning Rate Scheduler	ReduceLROnPlateau (factor=0.5, patience=2, mode='min')
DataLoader workers	2 (num_workers=2)
Max ảnh mỗi lớp	3.500 ảnh (cân bằng dataset)

Chiến lược *Early Stopping* được áp dụng để ngăn chặn hiện tượng quá khớp (*overfitting*): nếu giá trị lỗi trên tập kiểm định (`val_loss`) không giảm sau 5 epoch liên tiếp, quá trình huấn luyện sẽ dừng sớm và mô hình có kết quả tốt nhất đã lưu sẽ được sử dụng. Mô hình tối ưu nhất (`best_val_loss` thấp nhất) được lưu tự động vào Google Drive thông qua dòng lệnh:

```
torch.save(model.state_dict(),
           '/content/drive/MyDrive/best_model_adam.pth')
```

Hàm `train_model()` thực hiện ghi lại toàn bộ lịch sử huấn luyện bao gồm các chỉ số `train_loss`, `val_loss`, `train_acc`, và `val_acc` theo từng epoch, sau đó xuất ra file định dạng JSON để phục vụ cho các tác vụ phân tích và trực quan hóa dữ liệu về sau.

3.4 Triển khai module theo dõi và cấu hình logic cảnh báo

3.4.1 Tích hợp CNN với Deepsort

Class `PersonTrackerV2` là thành phần trung tâm của hệ thống, đảm nhiệm việc kết nối đầu ra phân loại của `MaskCNN` với thuật toán theo dõi DeepSORT. Quy trình tích hợp diễn ra trong hàm `process_frame()` theo các bước tuần tự sau:

- **Bước 1 — Phát hiện khuôn mặt:** MTCNN quét frame và trả về danh sách *bounding box* cùng điểm *confidence*. Chỉ các hộp có *confidence* ≥ 0.9 mới được giữ lại.
- **Bước 2 — Phân loại:** Vùng ảnh khuôn mặt được cắt ra, thực hiện thay đổi kích thước (*resize*) về 224×224 , chuẩn hóa bằng các thông số ImageNet (*mean/std*) rồi đưa qua mô hình `MaskCNN`. Hàm `argmax()` được sử dụng để chọn ra lớp (*class*) có xác suất cao nhất.

- **Bước 3 — Chuẩn hóa định dạng:** Bounding box từ định dạng ban đầu $xyxy$ được chuyển sang định dạng $xywh$ theo yêu cầu đầu vào của hàm `DeepSort.update_tracks()`, cụ thể là $[x1, y1, w, h]$.
- **Bước 4 — DeepSORT update:** Hàm `tracker.update_tracks(ds_input, frame=frame)` thực hiện thuật toán Hungarian (*Hungarian Algorithm*) để khớp kết quả phát hiện (*detection*) với các vết theo dõi (*track*) hiện có, đồng thời tạo *track* mới hoặc xóa bỏ các *track* cũ. Chỉ các *track* ở trạng thái Confirmed (duy trì liên tục trong `n_init=3` frame) mới được đưa vào xử lý ở các bước tiếp theo.

Cấu hình khởi tạo của DeepSort trong mã nguồn được thiết lập cụ thể như sau:

```
self.tracker = DeepSort(

    max_age=30, # Giữ track tối đa 30 frame khi mất detect

    n_init=3, # Cần 3 frame để xác nhận track mới

    nn_budget=100, # Số embedding tối đa lưu trong gallery

    max_iou_distance=0.7 # Ngưỡng IoU để khớp detection-track)
```

3.4.2 Xây dựng thuật toán đếm thời gian (Timer) theo ID đối tượng

Thuật toán đếm thời gian được thiết kế để theo dõi tích lũy thời gian đeo khẩu trang đúng cách cho từng cá nhân dựa trên Track ID. Mỗi `track_id` được quản lý bởi một dictionary trạng thái độc lập:

```
self.states = defaultdict(lambda: {

    "mask_start": None, # Thời điểm bắt đầu đeo đúng

    "mask_duration": 0.0, # Tổng thời gian tích lũy (giây)

    "last_class": None, # Trạng thái lần cuối

    "alerted": False, # Đã cảnh báo chưa
```

})

Logic đếm thời gian hoạt động theo nguyên tắc được thống kê chi tiết trong Bảng 3.7.

Bảng 3.7: Nguyên tắc hoạt động và xử lý logic của bộ đếm thời gian (Timer)

Trạng thái phát hiện (class_id)	Hành động của Timer
class_id == 1 (đeo đúng) và mask_start là None	Ghi nhận thời điểm bắt đầu: mask_start = current_time.
class_id == 1 (đeo đúng) và mask_start đã có	Cập nhật thời gian: mask_duration = current_time - mask_start.
class_id != 1 (đeo sai hoặc không đeo)	Reset hoàn toàn: mask_start = None, mask_duration = 0.0, alerted = False.
Track ID mất khỏi frame (max_age hết)	DeepSORT tự xóa track; states[tid] không bị xóa ngay (dùng defaultdict).

Đây là thiết kế quan trọng: đồng hồ sẽ được reset ngay khi người dùng tháo khẩu trang hoặc đeo sai, đảm bảo cảnh báo chỉ kích hoạt khi đối tượng đeo LIÊN TỤC quá ngưỡng thời gian quy định.

3.4.3 Cơ chế kích hoạt cảnh báo

Ngưỡng thời gian cảnh báo mặc định là ALERT_SECONDS = 10 giây, người dùng có thể tùy chỉnh tham số này trước khi khởi tạo bộ theo dõi đối tượng (*tracker*):

```
PersonTrackerV2.ALERT_SECONDS = alert_seconds # Ví dụ: 10, 20, 30 giây
```

```
tracker = PersonTrackerV2()
```

Cơ chế kích hoạt cảnh báo được kiểm tra tự động trong mỗi frame hình sau khi hệ thống cập nhật giá trị biến mask_duration:

```
alert = (cls_id != 1 and st['mask_duration'] > self.ALERT_SECONDS)
```

```
if alert and not st['alerted']:
```

```
    print(f'CẢNH BÁO: ID {tid} đeo {st["mask_duration"]:.1f}s')
```

```
st['alerted'] = True
```

Cờ hiệu alerted đảm bảo thông báo trên console chỉ in ra duy nhất một lần cho mỗi sự kiện vi phạm nhằm tránh hiện tượng rác log (*spam log*). Tác vụ trực quan hóa cảnh báo trực tiếp lên các frame video được đảm nhiệm bởi hàm tĩnh `PersonTrackerV2.draw()` với các thành phần cấu hình chi tiết trong Bảng 3.8.

Bảng 3.8: Các thành phần hiển thị trực quan và cấu hình màu sắc trên giao diện cảnh báo

Thành phần hiển thị	Màu sắc	Nội dung
Bounding box bình thường	Xanh lá (0,255,0)	Viền mỏng 2px — đang đeo đúng, chưa vượt ngưỡng.
Bounding box cảnh báo	Đỏ với viền đậm 4px	Vi phạm đang xảy ra.
Nhãn ID + trạng thái	Màu theo class (cam/xanh/đỏ)	”ID:5 With Mask” hiển thị phía trên bounding box.
Banner thời gian đeo	Xanh lá	”Thời gian đeo: 7.3s” hiển thị phía dưới bbox (chỉ khi class=1).
Banner cảnh báo đỏ	Nền đỏ bán trong suốt (60% opacity)	”!! CANH BAO: 12s !!” — xuất hiện khi vượt ngưỡng vi phạm.

Ngoài ra, hệ thống được thiết kế kiến trúc mở để mở rộng tích hợp âm thanh cảnh báo thông qua các thư viện chuyên dụng như `soundfile` hoặc `pygame`. Hiện tại, khi phát hiện vi phạm, cảnh báo mới chỉ được in ra màn hình console và hiển thị trực quan trên giao diện. Trong các phiên bản nâng cao, lệnh kích hoạt loa phát thanh có thể dễ dàng chèn vào khối lệnh điều kiện `if alert and not st['alerted']`: mà hoàn toàn không làm ảnh hưởng hay thay đổi cấu trúc tổng thể của hệ thống.

Mô-đun giao diện người dùng chính (`main_menu`) cho phép lựa chọn linh hoạt ba chế độ vận hành độc lập bao gồm:

- **Detect Image:** Xử lý ảnh tĩnh, không sử dụng bộ theo dõi đối tượng *tracker*.
- **Detect Video:** Xử lý tệp tin video đầu vào, tự động lưu kết quả đầu ra thành file `output_video.mp4`.
- **Realtime Webcam:** Kích hoạt và xử lý luồng dữ liệu camera trực tiếp theo thời gian thực.

Trong cả hai chế độ vận hành Video và Webcam, class `PersonTrackerV2` sẽ được khởi tạo mới với thông số `alert_seconds` mặc định là 10 giây, đồng thời chỉ số FPS hiện tại của hệ thống sẽ được hiển thị ở góc trên bên trái màn hình giúp người dùng dễ dàng giám sát hiệu năng.

4 TÀI LIỆU THAM KHẢO

- [1] Mohammadpour, M., et al. (2022). A Survey of Deep Learning Techniques for Object Tracking.
- [2] FPT AI. (2026). CCNN Documentation.